麒忆智脑 MOKOA 用户操作手册

版本 v3.3 · 最后修改时间 2026-09-15 · 面向演示操作与终端使用者

面向演示操作与终端使用者的图文操作手册

关于本手册

本手册面向两类读者：现场演示与验收的操作人员、日常使用本系统的终端使用者。它用尽量通俗的语言，**重点讲清网页界面上每一个看得见、点得着的元素怎么用**，不要求你具备编程或 AI 背景。需要搭建/调试环境的工程内容集中在安装与配置章节，可按需跳过。

符号	含义
提示	使用技巧或推荐做法
注意	容易踩坑或不可逆操作，请先阅读
备注	补充说明或技术背景（可跳读）
说明	能力边界或已验证结论

> **说明：** 本手册所有界面文案、按钮、字段、页签、状态文字均逐字对照前端源码（frontend/src/main.js、IngestionPanel.js）核对，与你看到的页面完全一致。端口、配置、API 也与后端源码一致。

目录

目录

1. 第一章　产品简介
2. 第二章　系统如何工作（通俗架构速览）
3. 第三章　安装与环境准备
4. 第四章　第一次使用（快速开始）
5. 第五章　界面总览与布局

6. 第六章　左栏对话区：与麒忆助理聊天
7. 第七章　右栏记忆面板：实时记忆
8. 第八章　多源接入面板：把资料交给记忆
9. 第九章　核心功能怎么用（结合界面）
10. 第十章　实用场景库
11. 第十一章　配置参考
12. 第十二章　命令与接口参考
13. 第十三章　隐私与数据安全
14. 第十四章　故障排查与常见问题
15. 第十五章　术语表
16. 附录 A　验收与表现摘要
17. 附录 B　文件与目录速览
18. 附录 C　版本与版权

第一章　产品简介

1.1　一句话理解麒忆智脑

麒忆智脑（MOKOA）= 一套运行在银河麒麟操作系统上的「智能体长期记忆」系统。它给 AI 助手装上记忆：你告诉它的偏好、你上传的文档与经验，都会被它记住，并在你下次对话时自动调用，让回答更贴合你本人。你通过网页与它交互——左边和助手对话，右边实时查看它「记住了什么」，全程无需编程。

1.2　为什么需要它

普通 AI 助手每次对话都是「失忆」的：上一轮你说「用中文、给可运行代码」，下一轮它又忘了。麒忆智脑把「记忆」作为独立能力抽出来：

没有记忆的助手	有麒忆智脑的助手
每次都重复说明偏好	偏好一次说完，长期生效
上传的资料用过即丢	资料沉淀为知识，随时被召回引用
敏感信息可能原样入库	手机号/密钥等写入即脱敏、加密
改主意后旧设定仍干扰	更新偏好自动留版本，旧版停用可查
想删某段记忆无从下手	用一句话即可精准遗忘、不可恢复

1.3　它能为你做什么

能力	你看得见、用得上的表现
记住偏好	你说「用中文回答、给可运行代码」，以后默认都这样；同一偏好更新会保留版本历史。
沉淀知识	上传 PDF / Word / 图片，或粘贴一段经验，它会学会并在问答中引用。
自动召回	每轮回答前自动调出相关记忆与偏好，界面显示检索耗时。
安全遗忘	说「忘掉……」即可删除指定记忆，删除后不可恢复。
隐私保护	手机号、邮箱、密钥等敏感信息在写入时自动脱敏打码，原文加密存储。
多源接入	支持文档、工具结果、用户行为、手动配置、对话五种来源统一入库。
跨场景隔离	不同场景的偏好互不污染，全局偏好与场景偏好分开放置。

1.4　设计理念

项目没有简单堆叠 Mem0 / MemoryOS / Cognee 等开源记忆库，而是**自研了一套「记忆编排核心（Memory Orchestrator）」**，把多源摄取、偏好捕捉、知识冲突处理、关联检索、短中长期流转、遗忘与脱敏统一编排起来，再参考三个开源项目的长处。

1.5　适用人群与场景

人群	典型场景
办公人员	固定邮件/文档风格、常用术语、项目背景记忆
开发者/运维	命令偏好、镜像源、报错经验、工具用法沉淀
系统操作员	「执行前先确认」「用中文」等安全与交互偏好
测试/验收同学	按指南在麒麟真机跑通主链与指标
演示讲解	左对话右记忆双栏，直观展示「边聊边学」

1.6 先记住几个关键词

词	通俗解释
记忆	系统为你保存的一切内容，分偏好、知识、行为等类型。
偏好（Preference）	你对助手提出的固定要求，如语言、风格、工具选择。
知识（Knowledge）	你上传或提供的经验、文档内容，供问答引用。
场景（Scene）	一次连续任务上下文，如「项目 A」「coding」。场景内偏好可隔离。
召回（Recall）	回答前自动找出与你当前问题相关的记忆。
短期/中期/长期	记忆按热度与重要性在三层间流转，越长期越「沉淀」。

> **备注：**看不懂这些词没关系，第五~八章会结合界面逐个演示；术语表（第十五章）也可随时回查。

1.7 隐私与数据自主权

麒忆智脑把「隐私」当作一等公民，而非事后补丁。三条原则贯穿设计：

原则	系统落地
数据不出端	敏感识别与脱敏在**本地**完成，不依赖第三方云服务做隐私判断
原文不落库	手机号/邮箱/密钥等写入即替换为加盐哈希占位符（如 `<HASH_7f3a9b>`），敏感原文加密存储
遗忘可精准	一句话即可按类型/关键词/时间范围精准删除，且多库级联、不可恢复

> **备注：**脱敏不覆盖所有隐式隐私（如「我住在 XX 小区」这类未命中正则的表述）。涉及高度敏感信息仍建议自行避免输入。

1.8 它和「通用记忆库 / 无记忆助手」的差异

对比对象	麒忆智脑的做法
无记忆的对话助手	把「记忆」抽成独立能力：偏好一次说完长期生效、上传资料沉淀为知识随时召回
直接堆叠 Mem0/MemoryOS/Cognee	自研编排核心统一调度，仅借鉴三者设计思路，避免功能重叠与答辩成本
黑盒记忆产品	每次入库/拒绝都返回可解释的**质量评分与原因**（quality_components / reasons），可调可查

1.9 能力边界与诚实声明

为避免误用，明确以下边界：

- 语义级敏感识别（LLM）代码已就绪，但**默认关闭**，按需通过 `security.semantic_detection.enabled` 开启；未开启时回退正则脱敏。
- 麒麟 SDK 端侧向量性能**在真机验收前不宣称已达标**；开发机已接入 BGE-M3 本地模型跑通主链。
- 偏好抽取支持「规则 / LLM / 融合」三模式；融合模式在 LLM 不可用时自动降级规则，保证离线可复现。
- 进程内存后端（`memory`）的延迟**不得**作为 ≤500ms 验收数据；验收须用真实向量后端。

第二章　系统如何工作（通俗架构速览）

2.1　数据是怎么流动的

你的一次输入（一句话、一个文件、一条经验）会经过下面这条流水线，最终变成「可召回的记忆」：

数据流转：采集→清洗→标准化→质量→去重 → 写入（短期→中期→长期） → 召回注入 → 遗忘级联擦除

2.2　五个核心环节

环节	系统在做什么	和你有关的部分
采集	从对话、上传、表单等多源收集原始内容	你说话、传文件、填表单
清洗/标准化	去噪、统一格式、切片	上传扫描件建议补一句说明
质量	对内容打分，低价值不入库	页面显示质量评分与原因
理解	判断是偏好还是知识，提取键与值	偏好自动成卡片并版本化
召回	按语义+关键词+别名找出相关记忆	回答区显示检索耗时与条数

2.3　记忆的三种形态

系统用三层记忆管理「冷热」：越常被用到、越重要的内容，越往长期沉淀。

形态	存放内容	你看得见的表现	流转规则
短期	当前会话刚产生的记忆	对话中即时出现	被多次访问且质量够高 → 升中期
中期	较活跃、经常复用的记忆	记忆/偏好页签常驻	抽象提炼后 → 升长期/知识库
长期/知识	沉淀的经验与文档	跨会话稳定可用	最稳定一层，召回优先命中

> **备注：**短期记忆默认容量约 10–20 条（热窗口），中期约 50 条，超过会按热度淘汰；中期记忆按「访问次数 + 质量×5 + 置信度」计算热度，低热度被驱逐。这些阈值可在配置中调整（见第十一章）。

2.4　一次输入的完整生命周期（时序）

以「在对话里说一句偏好」为例，后台依次发生：

1. **接入**：前端把消息连同 `user_id`、`scene_id` 发给 `/agent/chat`（或 `/ingest`）。
2. **采集 + 信封**：ingestion 校验来源、补齐用户/场景信封、记录 UTC 时间。
3. **清洗**：NFKC 标准化、去 ANSI 控制码、去不可见字符、超长截断。
4. **标准化**：转成统一 `NormalizedRecord` JSON（type / content / time / confidence / source …）。

5. **质量评分**：六维加权打分，低于阈值返回 `low_quality` 拒绝（带原因）。
6. **去重**：质量门控通过后才登记 SHA-256 指纹，TTL 窗口内重复合并。
7. **脱敏**：写入前调用 `filter_sensitive()`，敏感信息替换为占位符并加密索引。
8. **分类**：`classifier` 判断是偏好还是知识，提取键与值。
9. **写入**：进入短期记忆；命中冲突则 `conflict_resolver` 裁决（见 9.4）。
10. **晋升**：被多次访问 / 质量够高时，短期→中期→长期逐级沉淀。
11. **召回注入**：下一轮回答前 `recall()` 按语义 + 关键词 + 别名找出相关记忆，注入 Agent。

2.5 核心组件职责一览

组件（源码）	职责
`core/orchestrator.py`	总调度：process_input / classify / consolidate / recall / forget
`core/classifier.py`	偏好 vs 知识分类
`core/conflict_resolver.py`	新旧记忆撞车裁决（知识取最新事实；偏好看信任 + 时间）
`core/recall_ranker.py`	关联检索排序（语义 / 词面 / 别名 / 项目别名融合）
`core/layer_manager.py` + `short_term/mid_term`	短 / 中 / 长期三层管理与晋升
`core/context_perception.py`	上下文压缩，窗口达 70–80% 主动重构，用户无感
`ingestion/*`	多源采集 → 清洗 → 标准化 → 质量 → 去重
`preference/*`	偏好抽取、版本化、置信度、结构化画像
`security/*`	脱敏、NL 遗忘解析、索引 AES 加密
`embedding/*`	统一向量接口（kylin / local / hash）
`knowledge/memory_service/*`	向量存储与检索（Milvus / 麒麟向量）

2.6 记忆分层与流转参数

默认参数（均可在 `configs/default.yaml` 调整，详见第十一章）：

参数	默认	含义
短期容量	10 条	热窗口；超出按热度淘汰
中期容量	50 条	活跃复用记忆上限
晋升热度阈值	3.0	短期→中期所需累计热度
入库质量阈值	0.46	低于此值不写记忆
中期相似度门槛	0.7	中期去重 / 归并语义相似度
中期抽象提炼(LLM)	false	是否用大模型做中→长抽象

第三章　安装与环境准备

3.1　环境要求

项目	系统无关功能演示推荐	麒麟真机验收
操作系统	Windows 10/11、macOS 12+、Linux	银河麒麟桌面 V11（2603+）
Python	3.12（虚拟环境 .venv）	系统 Python 3.11/3.12
Node.js	18+（仅前端）	18+
内存	≥ 8 GB（本地嵌入模型占用较高）	≥ 8 GB
磁盘	≥ 4 GB（含可选 BGE-M3 模型）	≥ 4 GB
联网	首次安装依赖需联网	验收阶段按需联网

> **提示：**本机快速演示可用「hash 嵌入 + 内存存储」模式，**无需大模型、无需 GPU** 即可跑通主链路；正式验收再切换为麒麟向量引擎与麒麟 SDK。

3.2　方式一：本机快速演示（Windows / macOS / Linux）

步骤 1　准备 Python 环境

```
cd MOKOA
python -m venv .venv
# Windows 激活：
.venv\Scripts\activate
# macOS / Linux 激活：
source .venv/bin/activate
python -m pip install --upgrade pip
```

步骤 2　安装依赖

```
python -m pip install -r requirements.txt
```

步骤 3　（可选）配置对话模型

若想让助手真正「理解并回答」，在 `.env` 填入对话模型密钥；仅演示记忆链路可跳过，此时用规则模式跑通，不调用大模型。

```
# .env
AGENT_API_KEY=sk-xxxx
AGENT_MODEL=deepseek-ai/DeepSeek-V4-Flash
```

步骤 4　启动后端（记忆服务）

```
export OS_AGENT_EMBEDDING_BACKEND=hash    # 演示：无需 GPU
```

```
python -m uvicorn api.server:app --host 127.0.0.1 --port 18080
```

后端默认监听 **18080** 端口。看到 `Uvicorn running on http://127.0.0.1:18080` 即启动成功。

步骤 5　启动前端（网页界面）

```
cd frontend
npm install
npm run dev
```

浏览器访问 **http://localhost:5173** 即可看到对话界面（前端默认代理指向 18080）。

> **备注：** Windows 上若 `python` 未绑定，可用 `py` 或 `.venv\Scripts\python`；PowerShell 里 `export` 改为 `$env:OS_AGENT_EMBEDDING_BACKEND="hash"`。

3.3　方式二：银河麒麟 V11 真机部署（验收）

真机部署遵循「不破坏 OSTree 不可变系统」原则，分六个阶段逐步验证，前阶段未过不跳后阶段：

阶段	目标	通过标志
A 系统基线	记录系统/CPU/内存/OS 版本	留存 baseline 文本与截图
B 主链跑通	Python+FastAPI+记忆主链可运行	smoke_e2e 输出 passed:true
C 严格 Kylin 嵌入	真实 SDK 返回向量，不用 fallback 冒充	using_fallback=false，得到真实维度
D 接向量库	获得真实维度后连 Milvus/麒麟向量引擎	集合 schema 与维度匹配
E 前后端全链	页面真实读写记忆	右栏显示实时数据
F 指标报告	标准数据集跑分	召回/延迟/冲突/偏好达标

> **注意：** ⚠ 严禁为安装开发包擅自解除 OSTree 保护或破坏系统基线；`using_fallback=true` 时不得记为 SDK 通过；进程内存后端的延迟不得作为 500ms 验收数据。详细命令见仓库《测试同学_银河麒麟 V11 虚拟机部署与验收指南.md》。

3.4　方式三：Docker（可选）

仓库提供容器化入口时，可用镜像一键起后端。需在镜像内把 18080 端口映射出来，并把 `.env` 挂载进容器。前端仍可用宿主 `npm run dev` 或构建静态产物。

3.5　验证安装

```
curl -i http://127.0.0.1:18080/health
# 期望 HTTP 200 与 {"status":"ok", ...}
```

健康检查通过，说明后端服务、配置、存储后端均已就绪，可以打开前端连接使用了。

3.6 配置文件 `.env` 详解

后端敏感项与可切换项都通过 `.env`（或同名环境变量）注入，不写进 YAML。常用项：

变量	作用	示例
AGENT_API_KEY / AGENT_MODEL	对话模型密钥与模型名	sk-xxx / deepseek-ai/DeepSeek-V4-Flash
AGENT_BASE_URL	对话模型服务基址	https://api.siliconflow.cn/v1
OS_AGENT_EMBEDDING_BACKEND	向量后端（hash / local / kylin）	hash（演示免 GPU）
OS_AGENT_MEMORY_BACKEND	记忆存储后端（memory / milvus / kylin）	memory（演示）
OS_AGENT_KYLIN_ACCEPTANCE	验收严格模式开关	1（真机验收）
MOKOA_SECURITY_KEY	敏感索引加密密钥（AES-256）	任意口令，SHA256 派生
SILICONFLOW_API_KEY	偏好抽取 LLM 密钥	sk-xxx

> **注意：**密钥类字段只走环境变量；正文里出现 `sk-xxxx` 仅为占位示意，请替换为真实值。`.env.example` 列出全部可选项。

3.7 目录与数据准备

首次启动会在项目根自动创建以下目录（无需手动建）：

目录	用途
`./data/ingestion_staging.db`	SQLite 中转库（审计 + 去重），自动建表
`./data/preferences/`	偏好版本索引（本地 JSON）
`~/.mokoa/security.key`	敏感索引加密密钥文件（首次自动生成；Windows 不限制权限）

> **提示：**若要彻底重置本地记忆，停止服务后删除 `./data/` 与 `~/.mokoa/security.key` 再重启即可（注意：该操作会清除已存记忆，不可恢复）。

3.8 重装与清理

- **重装依赖**：`python -m pip install -r requirements.txt --upgrade`。
- **重置记忆库**：删除 `./data/` 目录（见 3.7）。
- **重置密钥**：删除 `~/.mokoa/security.key`；下次启动会用新密钥（旧加密索引将无法还原，自动降级为空串）。
- **前端重装**：`cd frontend && rm -rf node_modules && npm install`。

第四章　第一次使用（快速开始）

4.1　三步走通

以本地演示为例，先后启动「后端」与「前端」两个程序，再用浏览器打开页面：

步骤	动作	预期
1 启动后端	`uvicorn api.server:app --port 18080`	终端显示 running on 18080
2 启动前端	`cd frontend && npm run dev`	浏览器可开 5173
3 连接对话	打开页面 → 点连接 → 输入一句话	右栏出现真实记忆数据

4.2　打开页面并连接

1. 打开 `http://localhost:5173` 后，右上角连接按钮显示「演示数据模式」。点击该按钮（或先填好用户 ID 再点「↻ 同步」）连上本地后端。
2. 连接成功后按钮变为「实时 API 已连接」（高亮），右栏状态变为「实时同步」，面板开始出现真实数据。
3. 在左下输入框输入一句话，例如「以后请用中文回答，并优先给出可执行的步骤」，回车或点「发送 ↑」。

> **注意：** ⚠ 未连接后端时页面为「演示数据模式」，仅展示示例数据，不会真正写入记忆。要让记忆真实生效，务必先点连接。

4.3　第一次体验「记忆」

建议这样验证记忆真的在工作：

```
① 输入：「以后用中文回答，代码默认 Python，并加注释。」→ 保存本轮内容（默认开）
② 点「新对话」清空会话
③ 再问：「帮我写个读取文件的函数」
④ 开启「使用记忆」时，回答会自动用中文+Python+注释；关闭则不会。
```

> **提示：** 对比最直观：同一问题，开/关「使用记忆」开关，观察回答差异。

4.4　接入对话模型（让助手真能回答）

仅演示记忆链路可跳过模型；要让助手真正理解并回答，在 `.env` 填：

```
AGENT_API_KEY=sk-xxxx
AGENT_MODEL=deepseek-ai/DeepSeek-V4-Flash
AGENT_BASE_URL=https://api.siliconflow.cn/v1    # 可选，留空用供应商默认
```

填好后重启后端，连接前端即可对话。未填时系统走「规则模式」：仍能写入 / 召回记忆，但不调用大模型生成回答。

4.5 三步验证记忆真的生效

1. 连接后端后，左栏说「以后用中文回答，代码默认 Python 并加注释」→ 右栏「偏好」页签应即时出现 `response_language` / `programming_style` 卡片。
2. 点「新对话」清空会话，再问「写个读文件的函数」。开启「使用记忆」时，回答应为中文 + Python + 注释；关闭则不会。
3. 点「+ 多源接入」上传一个 `.docx` →「上传并写入记忆」；右栏「总览」知识数增加，之后相关提问能在「记忆」页签召回该文档内容。

> **说明：**若偏好页签不出现卡片，先确认「保存本轮内容」开关已开，且已填 `AGENT_API_KEY`（模型用于抽取偏好）。

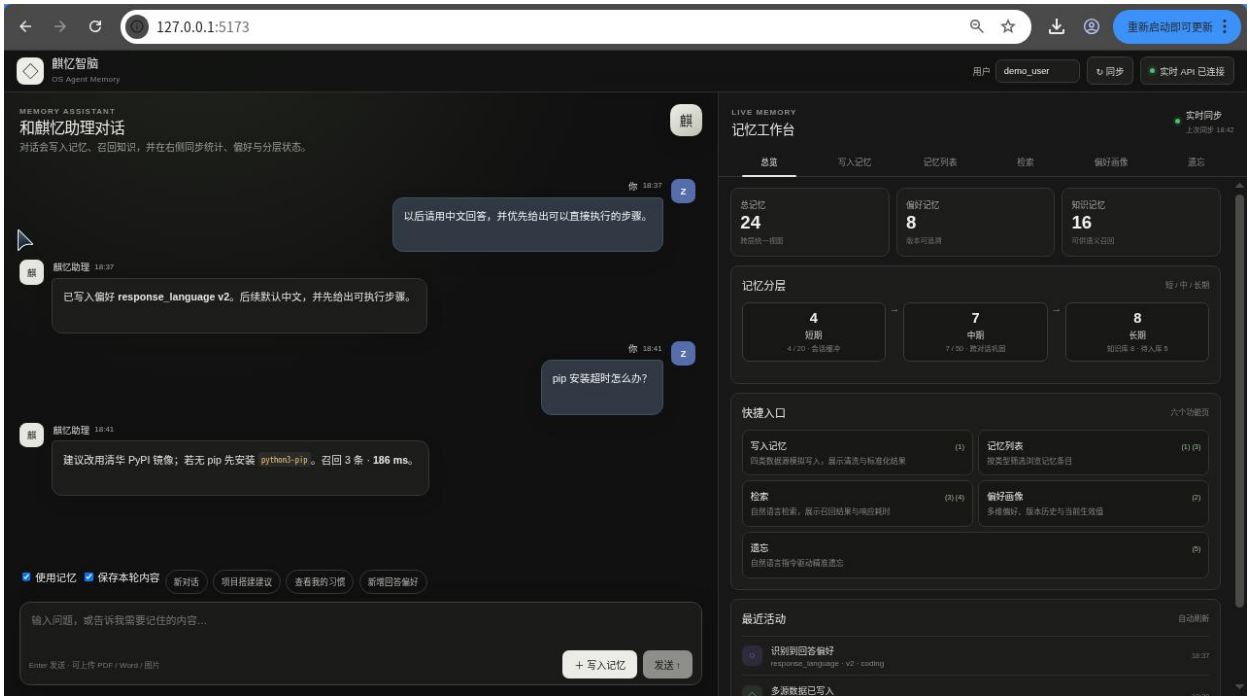
4.6 首跑常见卡点

卡点	原因	处理
后端起不来	依赖未装 / 端口占用	`pip install -r requirements.txt`；换端口或释放 18080
页面「演示数据模式」	未点连接	点右上角连接按钮，或填用户 ID 后点「↻ 同步」
偏好不入库	模型未配或开关未开	填 AGENT_API_KEY；确认「保存本轮内容」开
嵌入很慢	本地 BGE-M3 模型大	演示设 `OS_AGENT_EMBEDDING_BACKEND=hash`

第五章　界面总览与布局

5.1　整体布局：左右双栏

页面打开后是一个清晰的左右双栏布局，下面给出前端实际截图：





5.2　顶栏（Topbar）：连接与身份

顶栏位于页面最上方，负责「你是谁」和「是否连上后端」两件事。

元素	文案/状态	作用
品牌	◇ 麒忆智脑 · OS Agent Memory	项目标识
用户输入框	占位「用户」、aria 用户 ID	填写你的用户 ID（如 demo_user）；改完按回车或点同步
↻ 同步	同步中…/↻ 同步	切换或刷新当前用户的数据；连接态下重新拉取面板
连接按钮	演示数据模式 / 实时 API 已连接	核心开关：点它从演示切到真实后端，或断开回演示

说明：连接按钮亮起（高亮）表示已连上真实后端；显示「演示数据模式」且为灰色表示尚未连接。演示模式下你看到的是示例数据，操作不会真正写入记忆。

5.3 左栏与右栏的分工

区域	你在这里做什么	它在这里展示什么
左栏 对话区	和麒忆助理聊天、说偏好、发指令	助手回答（支持 Markdown）、工具调用过程
右栏 记忆面板	切页签查看/管理记忆、提交遗忘	总览/接入/偏好/记忆/遗忘 五类实时数据

5.4 状态指示：演示模式 vs 实时同步

右栏顶部有醒目的状态灯：

状态文字	含义	你能做什么
演示模式	未连接后端，显示示例数据	可点连接按钮切到实时；操作不落库
实时同步	已连接后端，数据真实	正常读写记忆；显示「上次同步」时间

备注：窄屏（窗口≤1050px）下，双栏会变为上下排列；此时点左栏「+ 多源接入」会自动滚动到右栏的接入页，方便你在手机/小窗里操作。

第六章　左栏对话区：与麒忆助理聊天

左栏是你和「麒忆助理」对话的地方。下面从上到下逐元素说明。

6.1　标题区

元素	文案	说明
眉标	MEMORY ASSISTANT	英文小字标识，无功能
主标题	和麒忆助理对话	对话区标题
副标题	对话会参与记忆召回，并在右侧实时形成偏好与记忆记录。	一句话说明左聊右记的关系
头像	麒	右侧圆形头像，代表「麒忆助理」

6.2　消息流（Chat Stream）

这里显示你和助手的每一轮对话，称为「消息行」。

元素	说明
头像	助手消息显示「麒」，你的消息显示「Z」
消息署名与时间	每条显示「麒忆助理 / 你」+ 时间（如 刚刚、14:32）
气泡内容	助手回答支持 Markdown 渲染（标题、列表、代码块、表格、链接均可正常显示）
工具步骤	若助手调用了工具，气泡下方展开步骤：工具名、参数、状态（运行中/完成/失败）、输出

工具步骤长什么样

当助手执行工具（如检索、写文件），你会看到一条带状态点的步骤：

状态点	文字	含义
运行中（橙）	运行中	工具正在执行
完成（绿）	完成	工具成功，下方显示输出
失败（红）	失败	工具出错，下方显示错误

示例：一条工具步骤可能显示「pip · {\"action\":\"install\"} · 完成」，并可展开 `pip install` 的真实输出。

等待回答时的提示

发送后，助手思考期间会显示动画气泡，文案有两种：

条件	文案
≤2.5 秒	正在检索记忆并生成回答…
>2.5 秒（模型较慢）	正在等待模型回答和记忆处理…

6.3 第一行开关与按钮

消息流下方第一排是三个开关/按钮，决定本轮对话是否读写记忆：

元素	默认	作用	点了会怎样
☑ 使用记忆	开	每轮回答前是否召回相关记忆	关掉后回答不再参考记忆，仅基于当前对话
☑ 保存本轮内容	开	本轮对话是否写入记忆	关掉后你说的内容不会被记忆（适合临时试探）
新对话	—	清空当前会话	清空所有消息与助手上下文；不影响已保存的记忆

> **提示：**演示时通常两个开关都保持开启。想「只聊不记」，可临时关掉「保存本轮内容」；想「只看不忆」，关掉「使用记忆」。

6.4 第二行快捷提问

消息流下方第二排是三个一键填入示例的按钮，点一下会把对应问题填进输入框（不自动发送，你可修改后发）：

按钮	填入的内容	用途
项目搭建建议	根据我的偏好，给出一个 Python 项目搭建方案	演示偏好如何影响方案
查看我的习惯	你记得我的哪些使用习惯?	主动查询已保存的偏好
新增回答偏好	以后回答请更加简洁	快速新增一条偏好

6.5 输入框与发送

元素	文案/行为	说明
输入框	占位「输入问题，或告诉我需要记住的内容…」	在此打字；Enter 发送、Shift+Enter 换行
提示文字	Enter 发送 · 可上传 PDF / Word / 图片	提醒快捷键与可上传类型
＋ 多源接入	按钮	打开右栏「接入」页签，去上传文件/填资料
发送 ↑	发送中/发送 ↑	点击或 Enter 发送；内容为空或正在处理时禁用

> **说明：**输入框不只是「问问题」——你在这里说的偏好、经验，只要在「保存本轮内容」开启时，都会被系统记住。所以「告诉我需要记住的内容…」这句提示是认真的：直接说偏好即可。

6.6 连接前的演示行为

未连接后端（演示数据模式）时，发送消息助手会给出示例式回答，且不会伪造「删除成功」等结果——它只会提示你「连接 API 后将显示真实召回与写入结果」。这是刻意设计：演示模式绝不假装数据已写入，避免误导。

第七章　右栏记忆面板：实时记忆

右栏是「记忆的可视化窗口」。顶部是状态，下面是五个页签。本章逐个说明。

7.1　面板头部与状态

元素	文案	说明
眉标	LIVE MEMORY	英文小字标识
标题	实时记忆面板	面板标题
状态灯	● 实时同步 / 演示模式	是否连后端；亮=实时
上次同步	上次同步 14:32	最近一次真实数据同步时间（演示模式显示「尚未连接」）

7.2　五个页签

页签栏有五个按钮：总览 / 接入 / 偏好 / 记忆 / 遗忘。点击切换，对应内容显示在下方。

页签	里面有什么（一句话）
总览	记忆总量、分层进度、最近活动，一眼掌握全貌
接入	提交多源内容的入口（文档/工具结果/行为/配置/对话）
偏好	当前生效的偏好卡片 + 完整版本历史
记忆	本轮对话实际召回的记忆卡片 + 检索耗时
遗忘	用自然语言删除记忆的入口

7.3　总览页签

打开即默认显示。分三块：

① 统计卡片（Stat Grid）

卡片	数字含义	副标题
总记忆	所有记忆条数	跨层统一视图
偏好记忆	偏好类条数	版本可追溯
知识记忆	知识类条数	可供语义召回

② 记忆分层进度条

四条进度条，直观看记忆在各层分布（数字为该层当前条数 / 容量）：

分层	含义	容量
短期记忆	当前会话热记忆	约 20
中期记忆	较活跃复用记忆	约 50
长期待入库	待抽象提炼沉淀	—

知识库	已沉淀可检索	—

③ 最近活动

一条滚动的活动流，记录每一次写入、更新、遗忘。每条含图标与文字：

图标	类型	示例
○	偏好	识别到回答偏好 · response_language v2
◇	记忆	沉淀项目上下文 · conversation long
⌫	遗忘	遗忘已处理，聊天上下文已清空
!	错误	记忆处理提示 / 调用失败

7.4 偏好页签

以卡片展示当前生效的偏好。每张卡片含：

卡片字段	含义
键（code）	偏好键名，如 response_language / programming_style
版本 vN	当前版本号；更新后递增
值（正文）	偏好的具体内容，如「代码默认 Python 并加注释」
页脚	场景（如 coding）+ 置信度百分比（如 94%）
准备遗忘	按钮：一键把该偏好填入遗忘页签，方便删除

卡片下方是「版本历史」区，列出该偏好所有版本：当前版本标注「当前版本」，旧版本标注「已被替换」，可追溯你何时改、改成什么。

7.5 记忆页签

显示本轮对话实际召回的记忆。标题处会显示「检索耗时 · 条数」（如 386 ms · 3 条）。每张记忆卡片含：

字段	含义
类型（code）	knowledge / preference 等
分层	short / mid / long 等
记忆编号	该条记忆的唯一 ID
正文	记忆内容摘要
页脚	场景 + 来源（如 conversation / document）
准备遗忘	按钮：一键填入遗忘页签删除该条

> **提示：**这是观察「它到底想起了什么」最直接的窗口。对比开启/关闭「使用记忆」时这里的条数，能直观感

> 受召回效果。

7.6 遗忘页签

用于安全删除记忆。

元素	文案	说明
标题区	SECURE FORGETTING / 安全遗忘	说明：用自然语言描述需删除的记忆，只有后端真实删除后右栏才更新
输入框	占位「例如：忘掉关于 Java 的偏好」	填写自然语言遗忘指令
执行遗忘	处理中…/执行遗忘	红色危险按钮；点击向后端发送遗忘指令
结果区	等待遗忘指令…/已处理…	显示删除数量与删除 ID；失败时显示原因
隐私提示	敏感信息由后端安全模块解析和清理；前端不会伪造成功结果。	强调不可逆与真实性

> **注意：** ⚠ 遗忘不可恢复：执行后内容在短期/中期/长期/中转库全部擦除，并清空当前会话。执行前请确认指令准确（如「关于 Java 的偏好」会语义匹配批量删除）。

第八章　多源接入面板：把资料交给记忆

点左栏「+ 多源接入」或右栏「接入」页签，即打开本面板。它把文档、图片、工具经验、使用习惯统一写入记忆。

8.1　面板头部与提示

元素	文案	说明
标题	多源数据接入 · 统一写入 · 实时反馈	面板标题与副标题
介绍	把文档、图片、工具经验和使用习惯交给记忆。处理结果会同步到对话和活动记录。	功能说明
演示提示	当前为演示模式。可以准备数据，连接 API 后才能提交；文件尚未上传。	未连接时提示

8.2　两类必填项：数据来源 / 场景

字段	说明
数据来源（下拉）	五选一：文档 / 工具结果 / 用户行为 / 手动配置 / 对话
场景（选填）	如 coding；用于场景隔离。留空即全局

8.3　来源一：文档（上传文件）

选「文档」时出现文件拖放区与待上传列表：

元素	说明
拖放区	「↥ 把文件拖到这里」+「选择文件」按钮；也可直接拖入
支持类型	PDF / Word(.docx) / 图片；每个 ≤ 8MB，可多选（最多 10 个）
内容说明（选填）	为每张文件补一句说明，如「截图中 pip 安装超时，切换镜像后解决」；扫描件尤其建议补
状态标签	待上传 / 处理中 / 文件不符合要求 / 已接收 / 未接收 / 请求异常
移除	从队列移除某个文件
修改后加入待上传队列	被拒/出错后可改完重新提交

> **备注：**文件不会在选中时立即上传；只有点底部「上传并写入记忆（N）」才发送。N 为当前待上传数量。

8.4　来源二：工具结果

记录一次工具执行的经验（如安装失败如何解决）：

字段	说明
工具名称	必填，如 pip

执行状态	下拉：失败 / 完成 / 超时
工具输出	必填，粘贴关键输出、报错及解决方式
事件编号（选填）	同一事件重复投递时用相同编号去重

8.5 来源三：用户行为

字段	说明
行为	必填，如 打开
操作对象	必填，如 VSCode
出现次数	必填，数字≥1，填真实次数
提示	单次行为可能因记忆价值不足而不被接收

8.6 来源四：手动配置

一次性固定设定偏好，等价于对话里说偏好：

字段	说明
配置键	必填，如 reply_language
配置值	必填，如 中文
作用范围	下拉：全局 / 当前场景

8.7 来源五：对话

直接粘贴一段希望记住的内容（必填），由系统判断是偏好还是知识。

8.8 提交与处理记录

底部「上传并写入记忆（N）」或「写入记忆」按钮：未连接后端、或未填必填项、或文档类无待上传文件时禁用。点击后显示进度「正在处理 XX / 正在等待处理结果…」；首次较慢会提示「模型初始化或内容解析中，请耐心等待」。

提交后，「本次会话处理记录」区逐条显示结果卡片：

卡片内容	含义
标签 + 状态	来源名 + 已接收/未接收/请求异常
说明文字	如「已接收并写入记忆」「已更新既有记忆」，未接收时给真实原因
标签组	来源 + 质量 X% + 处理 X ms（来自后端）
记忆编号	本条记忆的唯一 ID
原因代码	未接收时的 reject_code，便于排查
展开项	查看处理后的正文 / 质量评分明细（quality_components）
长期同步提示	知识类接受后是否进入长期记忆，可在对话中检索

说明：质量评分低（低于门控）或内容重复时，会显示「未接收」并给原因——这不是报错，是记忆策略在过滤低价值内容。可据此补充更有信息量的内容再投。

第九章　核心功能怎么用（结合界面）

9.1　偏好记忆：让助手记住你的习惯

怎么操作（三种入口）

入口	做法
对话输入	在左栏输入框说偏好，保存本轮内容开启 → 右栏「偏好」页签即时出现卡片
手动配置	接入页签选「手动配置」，填 配置键/值/作用范围
快捷提问	点「新增回答偏好」一键填入示例后发送

同一偏好被更新（如「代码改用 Go」）时，右栏偏好页签生成新版本 v2，旧 v1 在版本历史标注「已被替换」。动态键兜底保证长尾偏好不被静默丢弃（落为 habit.custom_*）。

9.2　知识沉淀：把资料变成可调用的记忆

用第八章的「多源接入」面板提交。后台经五段式处理（采集→清洗→标准化→质量→去重），质量达标才入库；处理记录区显示质量评分与原因。

9.3　自动召回：它怎么「想起来」

「使用记忆」开启时，每轮回答前自动检索。观察点：回答区下方、右栏「记忆」页签（含检索耗时与召回卡片）。召回综合语义相似、关键词/别名、意图识别三类信号。

9.3.1　检索模式 retrieval_mode

API `/recall`（及前端调用）支持三种模式，由 `retrieval_mode` 控制：

模式	行为	何时用
auto（默认）	中英文智能选路：英文开放域问句直接走**语义排序**（跳过词面全表扫描，P95≈386ms）；命中已知项目别名 / 办公短语的英文放行混合召回	日常对话，兼顾速度与覆盖
semantic	强制语义排序，不做词面补召	纯语义意图、中英皆可
hybrid	语义 + 词面补召双路融合	项目别名/专名未进向量候选时保召回

> **说明：** auto 模式下英文查询默认走语义路径，正是修复「词面全表扫描导致 P95 破 500ms」的关键；词面补召仅对确实命中已知项目的英文放行，避免再拖慢。

9.4　冲突自动消解

冲突	裁决	界面表现

知识冲突	知识取最新事实	旧值被取代，偏好页签不再显示
偏好冲突	来源信任 + 时间近因	更新的、更高信任的胜出，旧版进版本历史
场景隔离	不同场景互不覆盖	切场景后对应偏好生效

9.4.1 知识冲突：取最新事实

知识类记忆以「事实」为准，不盲信高信任旧来源。规则（见 `conflict_resolver.resolve_knowledge`）：

- 新事实**更新鲜** → 默认 `REPLACE`（以最新为准）；
- 新事实**不更新鲜** → `KEEP_OLD`；
- 仅当**低信任观测**（behavior / tool_result）试图覆盖**更高信任旧事实**（conversation / manual_config）时，才保留旧值。

> **说明：示例**：手动配置「端口 8080」，之后对话给出更新鲜的「端口 9090」→ 取 9090。这正是修复「手动配置永远压过对话」后冲突正确率达 100% 的关键。

9.4.2 偏好冲突：信任分 + 近因

各类来源有固定信任分（见 `conflict_resolver`）：

来源	信任分
manual_config（手动配置）	3
conversation（对话）	2
behavior / tool_result（行为 / 工具结果）	1

裁决逻辑：新来源信任分更高 → 胜出；信任分相同且新事实更新鲜 → 胜出（REPLACE）；否则保留旧版（KEEP_OLD）。同一偏好的旧版本进「版本历史」，可追溯。

9.5 短中长期流转

记忆按热度在三层间流动（见 2.3）。右栏「总览」的分层进度条实时反映分布；服务重启后记忆可恢复。

9.6 安全遗忘

两种入口：对话内以「忘掉/忘记/删除/清除」开头直接触发；或右栏「遗忘」页签填自然语言点「执行遗忘」。删除不可逆（见 7.6）。记忆卡片与偏好卡片上的「准备遗忘」按钮可一键把目标填入遗忘页签。

9.7 隐私保护

写入时手机号/邮箱/密钥等约 58 条正则脱敏、AES 加密；遗忘页签的隐私提示明确「前端不会伪造成功结果」。涉及高度敏感信息请自行避免输入。

9.8 偏好抽取三模式与降级

偏好抽取支持三种运行模式（由 `preference.*` 配置，详见 11.14）：

模式	配置	行为
规则	use_llm=false, allow_rules=true	只走规则抽取，离线可复现
LLM	use_llm=true, allow_rules=false	只走大模型；失败则空结果（无规则兜底）
融合（默认）	use_llm=true, allow_rules=true	先 LLM；无密钥 / 超时 / 空结果时自动降级规则；规则补漏长尾

> **提示：** 默认「融合」模式最稳——联网时用大模型抓长尾表达，断网或超时自动回规则，不漏掉明确偏好。实测精确率 100% / 召回率 95.83%（规则模式，35 条金标准）。

9.9 动态键兜底（长尾偏好不丢）

当一条偏好无法归入已有标准键（如 `response_language`、`programming_style`）时，系统不会静默丢弃，而是自动归类到 `habit.custom_*` 命名空间（例如 `habit.custom_coding_language`）；类别也不合法时兜底到 `habit.custom_{话题}`。这样任意长尾习惯都能被记住、被召回、可被遗忘指令命中。

> **说明：** 标准偏好类别共 6 类：`habit`（习惯）、`style`（风格）、`security`（安全策略）、`tool_selection`（工具选择）、`workflow`（工作流）、`communication`（沟通）。`habit` 之外的新类别也走 `habit.custom_*`。

第十章　实用场景库

下面场景都可在界面照做。每个给出「点哪里 / 说什么 / 看什么」。

场景一 · 办公邮件助手

连接后端 → 对话输入「回复邮件要简洁，默认用中文」→ 保存本轮内容开启 → 右栏「偏好」出现卡片。后续问「帮我起草请假邮件」，回答自动简洁中文。

场景二 · 编程 Agent

对话输入「pip 安装超时改用国内镜像」「要可运行代码」→ 再问「怎么装这个依赖？」→ 右栏「记忆」页签可见召回的镜像偏好，回答给出可执行命令。

场景三 · 系统操作安全偏好

对话设「命令执行前先确认」→ 后续涉及执行的操作，助手先征求同意。

场景四 · 项目背景沉淀

点「＋ 多源接入」→ 来源选「文档」拖入 项目说明.docx → 填内容说明 → 点「上传并写入记忆」→ 处理记录显示「已接收并写入记忆」。问「项目怎么跑」时助手结合文档回答。

场景五 · 跨场景偏好隔离

接入页签手动配置：场景 coding 设「用 Go」、场景 os_operation 设「用 Python」；切换场景提问，回答语言随场景切换。

场景六 · 改主意并保留历史

先说「代码用 Python」，再说「改一下，用 Go」→ 偏好页签出现 v1(Python,已被替换)、v2(Go,当前版本)。

场景七 · 精准遗忘

对话说「忘掉关于镜像的偏好」（直接触发），或右栏「遗忘」页签填「忘掉关于镜像的偏好」点「执行遗忘」→ 结果区显示删除数量与 ID；重新查记忆确认消失。

场景八 · 隐私打码验证

对话输入含手机号示例 → 保存后到偏好/记忆页签查看，号码已脱敏为占位符。

场景九 · 工具经验复用

接入页签选「工具结果」：工具名称 pip、状态 失败、输出「连接超时，切换镜像后成功」→ 写入。下次相关提问召回该经验。

场景十 · 行为记忆

接入页签选「用户行为」：行为 打开、对象 VSCode、次数 5 → 写入。系统据真实次数判断记忆价值。

10.1　端到端操作演练（推荐照做）

```
1) 点右上角连接按钮 → 状态变「实时 API 已连接」
2) 左栏输入：「以后用中文，代码默认 Python 并加注释」→ 发送
3) 右栏切「偏好」→ 看到 response_language / programming_style 卡片
4) 点「＋ 多源接入」→ 选「文档」拖入一个 .docx → 上传并写入记忆
5) 右栏切「总览」→ 总记忆/知识数增加，分层进度条变化
6) 点「新对话」→ 问「写个读文件函数」→ 右栏「记忆」显示召回，回答用中文+Python
7) 右栏「遗忘」→ 填「忘掉关于 Python 的偏好」→ 执行遗忘 → 确认已删
```

第十一章　配置参考（完整版）

本章逐条说明项目**唯一**的配置文件 `configs/default.yaml` 中每一个可配置项：它的**配置路径**（YAML 中的点分键）、**默认值**、可被哪个**环境变量**覆盖、有哪些**可选值**，以及每一项**含义与对系统行为的影响**。所有默认值均逐字对照源码 `configs/default.yaml` 与加载器 `src/common/config.py`。

11.0　配置从哪读取、优先级如何

- **唯一配置文件**：`configs/default.yaml`（仓库 `configs/` 目录下）。
- **加载器**：`src/common/config.py` 启动时读取该 YAML，对外提供 `CONFIG.get(path, default)`、`get_int`、`get_float`、`get_bool`。上层业务一律通过 `CONFIG.get("a.b.c")` 取配置，不直接读文件。
- **三层优先级（高 → 低）**：① 环境变量（最高，且只有环境变量能覆盖密钥类字段） ② `default.yaml` 中的值 ③ 代码内置兜底默认值。
- **密钥原则**：API Key、令牌等敏感字段**只允许通过环境变量注入**，绝不写在 YAML 里。

11.1　项目信息 `project`

配置路径	默认值	环境变量	可选值	含义与效果
project.name	mokoa	—	任意	项目内部标识，用于日志与路径拼接
project.display_name	麒忆智脑	—	任意	前端/文档展示名
project.version	0.1.0	—	语义化版本	与发布版本对应，影响 `/health` 返回

11.2　服务接口 `api`

配置路径	默认值	环境变量	可选值	含义与效果
api.host	127.0.0.1	—	IP / 域名	后端监听地址；验收常改 `0.0.0.0` 对外暴露
api.port	8000	—	端口号	服务端口；演示常用 `18080`（CLI/环境变量覆盖）
api.title	OS Agent Memory API	—	任意	OpenAPI 文档标题
api.version	0.3.0	—	任意	API 版本号，出现在 `/health`
api.cors_allow_origins	["*"]	API_CORS_ORIGINS	逗号分隔列表	允许跨域的前端源；生产应限定具体域名而非 `*`

11.3　记忆分层与质量 `memory`

配置路径	默认值	环境变量	可选值	含义与效果
memory.short_term_capacity	10	—	正整数	短期记忆容量（热窗口条数），超出按热度淘汰
memory.mid_term_capacity	50	—	正整数	中期记忆容量，超出按热度驱逐
memory.mid_term_heat_threshold	3.0	—	浮点	升入长期所需的累计热度阈值
memory.mid_term_similarity_threshold	0.7	—	0–1	中期去重/归并的语义相似度门槛
memory.mid_term_keyword_alpha	0.3	—	0–1	热度计算中关键词权重（其余为语义）
memory.mid_term_recency_tau_hours	168.0	—	浮点	近因衰减时间常数（小时），越大越"念旧"
memory.mid_term_recency_gamma	1.0	—	浮点	近因衰减指数
memory.mid_term_llm_abstraction	false	—	true / false	是否用大模型做中期→长期的抽象提炼
memory.quality_threshold	0.46	—	0–1	入库总质量门控；低于此值不写入记忆
memory.quality_type_thresholds.conversation	0.44	—	0–1	对话类内容的入库质量阈值
memory.quality_type_thresholds.document	0.51	—	0–1	文档类内容的入库质量阈值
memory.quality_type_thresholds.tool_result	0.54	—	0–1	工具结果类的入库质量阈值
memory.quality_type_thresholds.behavior	0.41	—	0–1	用户行为类的入库质量阈值
memory.quality_type_thresholds.manual_config	(fallback 0.46)	—	0–1	手动配置类未单独列出时回退到 `quality_threshold`
memory.quality_weights.source_reliability	0.25	—	0–1	质量分权重：来源可信度
memory.quality_weights.explicitness	0.20	—	0–1	质量分权重：显式程度
memory.quality_weights.recurrence	0.20	—	0–1	质量分权重：复现频次
memory.quality_weights.informativeness	0.15	—	0–1	质量分权重：信息量
memory.quality_weights.reuse_value	0.10	—	0–1	质量分权重：可复用价值
memory.quality_weights.input_confidence	0.10	—	0–1	质量分权重：输入置信度（六项合计 1.0）

> **备注：**质量分由各 `quality_weights` 加权求和得到，再与对应类型的 `quality_type_thresholds` 比较决定是否入库；手动配置未设独立阈值时回落到 `quality_threshold`。

11.4 检索 `retrieval`

配置路径	默认值	环境变量	可选值	含义与效果
retrieval.top_k	5	—	正整数	每轮召回返回的记忆条数上限
retrieval.latency_budget_ms	500	—	毫秒	检索阶段延迟预算（赛题 ≤500ms 硬指标）

11.5 嵌入（向量化）`embedding`

配置路径	默认值	环境变量	可选值	含义与效果
embedding.backend	auto	OS_AGENT_EMBEDDING_BACKEND / EMBEDDING_BACKEND	auto / kylin / local / hash	向量化后端：`hash` 演示免 GPU；`local` 本地 BGE-M3；`kylin` 麒麟 SDK；`auto` 自动选
embedding.local_model	BAAI/bge-m3	—	HuggingFace 模型名	`local` 后端使用的本地嵌入模型
embedding.normalize	true	—	true / false	是否对向量做 L2 归一化
embedding.allow_stub_fallback	true	—	true / false	后端不可用时是否允许降级桩（验收须设为 false）

11.6 存储 `storage`

配置路径	默认值	环境变量	可选值	含义与效果
storage.memory_backend	auto	OS_AGENT_MEMORY_BACKEND	auto / memory / milvus / kylin	记忆存储后端；`auto` 见 11.18 解析规则
storage.milvus_uri	http://localhost:19530	OS_AGENT_MILVUS_URI	URL	Milvus 向量库地址
storage.collection_name	os_agent_memory	OS_AGENT_MILVUS_COLLECTION / MILVUS_COLLECTION_NAME	集合名	记忆向量集合名
storage.staging_backend	sqlite	—	sqlite / jsonl	中转库类型（写入前暂存）
storage.staging_sqlite	./data/ingestion_staging.db	—	路径	SQLite 中转库文件位置
storage.staging_jsonl	./data/staging.jsonl	—	路径	JSONL 中转落盘位置
storage.staging_raw_archive_mode	redacted	—	redacted / off / plaintext	原始报文归档：`redacted` 脱敏后存；`off` 不存；`plaintext`

配置路径	默认值	环境变量	可选值	含义与效果
				原文存（敏感！）
storage.staging_allow_plaintext_raw_archive	false	—	true / false	是否允许 `plaintext` 归档模式
storage.staging_raw_retention_days	7	—	天数	原始报文保留天数
storage.staging_audit_retention_days	90	—	天数	审计日志保留天数
storage.staging_auto_purge_interval_seconds	3600	—	秒	中转库自动清理间隔
storage.dedup_ttl_seconds	300	—	秒	去重时间窗（窗口内重复内容合并）

11.7 麒麟向量 `kylin`

配置路径	默认值	环境变量	可选值	含义与效果
kylin.acceptance	false	OS_AGENT_KYLIN_ACCEPTANCE	true / false	正式验收开关；true 时严格模式，禁用降级冒充
kylin.strict	false	OS_AGENT_KYLIN_STRICT	true / false	严格模式；true 时禁用 fallback，必须拿到真实向量维度
kylin.vector_dim	1024	MEMORY_VECTOR_DIM	整数	向量维度；需与麒麟 SDK 实际维度一致
kylin.milvus_collection	mokoa_kylin_sdk	—	集合名	麒麟向量引擎集合名
kylin.vector_library	""	OS_AGENT_KYLIN_VECTOR_LIBRARY	路径	麒麟向量库（.so/.dll）路径；严格模式必填
kylin.vector_endpoint	""	OS_AGENT_KYLIN_VECTOR_ENDPOINT	地址	麒麟向量服务地址；真机连接用
kylin.vector_timeout_ms	5000	OS_AGENT_KYLIN_VECTOR_TIMEOUT_MS	毫秒	麒麟向量调用超时

11.8 知识服务 `knowledge.memory_service`

配置路径	默认值	环境变量	可选值	含义与效果
knowledge.memory_service.save_threshold	0.46	—	0–1	知识写入质量门槛
knowledge.memory_service.similarity_threshold	0.7	—	0–1	知识冲突判定的相似度门槛
knowledge.memory_service.conflict.enabled	true	—	true / false	是否启用知识冲突自动消解
knowledge.memory_service.conflict.near_recency_decay	true	—	true / false	近因衰减是否在冲突裁决中生效
knowledge.memory_service.text_max_length	4096	—	整数	单条记忆正文最大长度
knowledge.memory_service.short_text_max_length	512	—	整数	短文本最大长度
knowledge.memory_service.id_max_length	128	—	整数	记忆 ID 最大长度

11.9 摄取 `ingestion`

配置路径	默认值	环境变量	可选值	含义与效果
ingestion.vision	auto	—	auto / on / off	多模态视觉接入：`auto` 探测可用时开启；`on` 强制；`off` 关闭
ingestion.vision_probe	false	—	true / false	启动时是否主动探测视觉能力

11.10 智能体 `agent`

配置路径	默认值	环境变量	可选值	含义与效果
agent.provider	siliconflow	AGENT_PROVIDER	siliconflow / lm-studio / openai	对话模型供应商
agent.base_url	""	AGENT_BASE_URL	URL	模型服务基址（留空用供应商默认）
agent.model	""	AGENT_MODEL	模型名	对话模型名（如 deepseek-ai/DeepSeek-V4-Flash）
agent.api_key	""	AGENT_API_KEY	密钥	对话模型 API Key（仅环境变量）
agent.timeout_seconds	60.0	—	秒	对话模型调用超时
agent.tools_enabled	false	—	true / false	是否启用工具调用
agent.save_memory_default	true	—	true / false	新对话默认是否"保存本轮内容"
agent.use_memory_default	true	—	true / false	新对话默认是否"使用记忆"
agent.proxy_ignore_schemes	[socks://]	—	列表	走代理时忽略的协议

配置路径	默认值	环境变量	可选值	含义与效果
				(如 socks)

11.11 兼容适配 `adapters`

配置路径	默认值	环境变量	可选值	含义与效果
adapters.memoryos	true	—	true / false	兼容 MemoryOS 适配（参考/预留）
adapters.mem0	true	—	true / false	兼容 Mem0 适配（参考/预留）
adapters.cognee	true	—	true / false	兼容 Cognee 适配（参考/预留）

> **备注：** `adapters` 仅作开源方案对照与接口预留，不影响自研 Orchestrator 的主链路。

11.12 语言模型 `llm`

配置路径	默认值	环境变量	可选值	含义与效果
llm.provider	lm-studio	—	lm-studio / openai	底层 LLM 供应商
llm.base_url	http://127.0.0.1:1234	LM_STUDIO_BASE_URL / OPENAI_BASE_URL	URL	LLM 服务基址
llm.model	phi-3.1-mini-128k-instruct	LM_STUDIO_MODEL	模型名	LLM 模型名
llm.api_key	""	LM_STUDIO_API_KEY / OPENAI_API_KEY	密钥	LLM Key（仅环境变量）
llm.timeout_seconds	60.0	—	秒	LLM 调用超时
llm.max_tokens	1024	—	整数	单次生成最大 token 数

11.13 安全 `security`

配置路径	默认值	环境变量	可选值	含义与效果
security.enable_pii_filter	true	—	true / false	是否开启 PII 脱敏（手机号/邮箱/密钥等约 58 条正则）
security.enable_nl_forget	true	—	true / false	是否开启自然语言遗忘解析
security.semantic_detection.enabled	false	—	true / false	语义级敏感检测（默认关，按需开）
security.semantic_detection.min_confidence	0.6	—	0–1	语义检测命中最低置信度

11.14 偏好 `preference`

配置路径	默认值	环境变量	可选值	含义与效果
preference.storage_dir	./data/preferences	—	路径	偏好存储目录
preference.use_llm	true	—	true / false	是否用大模型辅助偏好抽取
preference.allow_rules	true	—	true / false	是否允许规则模式抽取（长尾兜底 `habit.custom_*`）
preference.llm.base_url	http://127.0.0.1:1234/v1	SILICONFLOW_BASE_URL	URL	偏好抽取 LLM 基址
preference.llm.model	phi-3.1-mini-128k-instruct	SILICONFLOW_LLM_MODEL / PREFERENCE_LLM_MODEL	模型名	偏好抽取模型
preference.llm.api_key	""	SILICONFLOW_API_KEY	密钥	偏好抽取 Key（仅环境变量）

11.15 日志 `logging`

配置路径	默认值	环境变量	可选值	含义与效果
logging.level	INFO	—	DEBUG / INFO / WARNING / ERROR	日志级别；排障可临时调 DEBUG
logging.format	(含时间/级别/模块)	—	格式串	日志行格式

11.16 评测 `benchmark`

配置路径	默认值	环境变量	可选值	含义与效果
benchmark.dataset_dir	./benchmarks/datasets	—	路径	评测数据集目录
benchmark.results_dir	./data/reports	—	路径	评测报告输出目录
benchmark.seed	42	—	整数	随机种子，保证评测可复现

11.17 环境变量覆盖总表

下表汇总所有可由环境变量覆盖的配置项（与 `src/common/config.py` 的 `ENV_OVERRIDES` 一致）：

环境变量	对应配置路径	类型
OS_AGENT_MEMORY_BACKEND	storage.memory_backend	字符串
OS_AGENT_MILVUS_URI	storage.milvus_uri	字符串
OS_AGENT_MILVUS_COLLECTION / MILVUS_COLLECTION_NAME	storage.collection_name	字符串

环境变量	对应配置路径	类型
OS_AGENT_EMBEDDING_BACKEND / EMBEDDING_BACKEND	embedding.backend	字符串
OS_AGENT_KYLIN_ACCEPTANCE	kylin.acceptance	布尔
OS_AGENT_KYLIN_STRICT	kylin.strict	布尔
OS_AGENT_KYLIN_VECTOR_LIBRARY	kylin.vector_library	字符串
OS_AGENT_KYLIN_VECTOR_ENDPOINT	kylin.vector_endpoint	字符串
OS_AGENT_KYLIN_VECTOR_TIMEOUT_MS	kylin.vector_timeout_ms	整数
MEMORY_VECTOR_DIM	kylin.vector_dim	整数
LM_STUDIO_API_KEY / OPENAI_API_KEY	llm.api_key	密钥
LM_STUDIO_BASE_URL / OPENAI_BASE_URL	llm.base_url	字符串
LM_STUDIO_MODEL	llm.model	字符串
AGENT_BASE_URL	agent.base_url	字符串
AGENT_MODEL	agent.model	字符串
AGENT_API_KEY	agent.api_key	密钥
SILICONFLOW_API_KEY	preference.llm.api_key	密钥
SILICONFLOW_BASE_URL	preference.llm.base_url	字符串
SILICONFLOW_LLM_MODEL / PREFERENCE_LLM_MODEL	preference.llm.model	字符串

说明： 另有 `API_CORS_ORIGINS`、`API_AUTH_TOKEN`、`API_MAX_BODY_BYTES` 等为生产加固常用的环境变量，由 `api/server.py` 直接读取；密钥类一律走环境变量，不进 YAML。

11.18 `auto` 的自动解析规则

`memory_backend` 与 `embedding.backend` 设为 `auto` 时，运行期按以下规则选择真实后端：

- **storage.memory_backend = auto**：

1. 若处于测试（`pytest`）环境 → `memory`（进程内，不落盘）；
2. 若在银河麒麟 V11 用户态且可加载向量 SDK（`vector_library` 已配置）→ `kylin`；
3. 否则 → `memory`。

注意： 显式填写 `memory` / `milvus` / `kylin` 时直接使用；填非上述值会报错。验收阶段须拿到真实向量维度，严禁用 `memory` 后端延迟冒充 500ms 指标。

- **embedding.backend = auto**：按运行环境在 `kylin` / `local` / `hash` 间择优；演示无 GPU 时可显式设 `hash` 免模型依赖。

11.19 推荐配置组合

场景	关键配置

本机快速演示	`OS_AGENT_EMBEDDING_BACKEND=hash` + `OS_AGENT_MEMORY_BACKEND=memory`，对话模型填 `AGENT_API_KEY/AGENT_MODEL`（或不填走规则模式）
银河麒麟真机验收	`OS_AGENT_KYLIN_ACCEPTANCE=1`，配齐 `kylin.vector_library/vector_endpoint`，`embedding.backend=kylin`，`storage.memory_backend=kylin`，`staging_raw_archive_mode=redacted`

说明：本机演示最小配置：只设 `AGENT_API_KEY/AGENT_MODEL` + `OS_AGENT_EMBEDDING_BACKEND=hash` + `OS_AGENT_MEMORY_BACKEND=memory` 即可跑通主链路。正式验收须切换为麒麟向量引擎与麒麟 SDK（`OS_AGENT_KYLIN_ACCEPTANCE=1`），且 `kylin.strict=true` 下禁用 fallback 冒充。

第十二章　命令与接口参考

后端基于 FastAPI（`api/server.py`），所有接口以 JSON 通信，默认监听 `18080`。本章给出面向使用 / 联调最常用、最核心的端点与字段；完整 OpenAPI 文档见服务启动后的 `/docs`。

12.1　常用命令

```
# 启动后端（演示：hash 嵌入 + 内存存储，免 GPU）
export OS_AGENT_EMBEDDING_BACKEND=hash
python -m uvicorn api.server:app --host 127.0.0.1 --port 18080
# 健康检查
curl -i http://127.0.0.1:18080/health
# 端到端冒烟测试（演示模式）
python scripts/smoke_e2e.py
# 回归测试
python -m pytest -q tests/
# 偏好抽取离线评测
python scripts/pref_eval_dataset.py --mode rule
# 中转库手动清理（7 天 raw / 90 天审计）
curl -X POST http://127.0.0.1:18080/staging/purge -H 'Content-Type: application/json' -d
'{}'
```

12.2　端点总览

端点	方法	作用	关键请求/返回
`/health`	GET	健康检查	返回 `status=ok`、各后端标识
`/agent/chat`	POST	同步对话 + 记忆写入/召回	见 12.3
`/agent/chat/stream`	POST	SSE 流式对话	逐事件推 token / 工具 / 答案
`/stats/{user_id}`	GET	右栏总览统计与分层	返回短/中/长/Milvus 四层 count/capacity
`/ingest`	POST	非文档类多源写入	见 12.4
`/ingest/file`	POST	文档/图片上传写入	multipart：file + user_id + scene_id? + caption?
`/short-term/{user_id}`	GET	短期记忆列表	limit / scene_id / memory_type 过滤
`/short-term/{user_id}/{memory_id}/touch`	POST	记录一次访问（升热度，促晋升）	返回 access_count / heat
`/mid-term/{user_id}`	GET	中期记忆列表	含 avg_heat
`/consolidate/{user_id}/mid`	POST	短期→中期整理	promoted_to_mid / short_remaining
`/consolidate/{user_id}`	POST	全量整理（短→中→长）	promoted_to_mid / promoted_to_long / stored_memory_ids
`/store-entry`	POST	直接写一条记忆（带	memory_id / created /

端点	方法	作用	关键请求/返回
		MemoryEntry)	superseded_memory_id
`/preferences/{user_id}`	GET	偏好列表	latest_only / top_k / preference_key 过滤
`/preferences/extract`	POST	从文本抽偏好并写库	见 12.5
`/preferences/{user_id}/profile`	GET	结构化偏好画像	以权威长期存储为准
`/preferences/{user_id}/history`	GET	偏好版本链	按 preference_key 查全部版本
`/recall`	POST	检索记忆	见 12.6
`/forget`	POST	自然语言遗忘，级联擦除	见 12.7
`/staging/stats`	GET	中转库拒绝码分布 / 耗时分位	按 user_id? 过滤
`/staging/purge`	POST	中转库保留期清理	raw_retention_days / audit_retention_days

12.3 /agent/chat 请求与返回（节选）

前端发送：`user_id`、`message`、`history`（最近 20 轮）、`use_memory`、`save_memory`。返回含 `answer`、`memories`（本轮召回）、`recall_ms`（检索耗时）、`latency_ms`（总耗时）、`memory_write.status`（accepted / rejected / disabled / failed / skipped）、遗忘时含 `deleted_ids`、`reset_history`。

> **备注：**消息以「忘掉/忘记/删除/清除」开头会在对话内直接触发遗忘，不调用模型生成，并清空当前会话。

12.4 /ingest（多源写入）请求与返回

请求体 `IngestRequest`：

字段	类型	说明
user_id	string	用户标识（必填；缺失时空兼容为 `default`）
source	string	来源：tool_result / user_behavior / manual_config / conversation / document
payload	object	来源专属内容（如 `action`/`target`/`content`/`key`/`value`）
scene_id	string?	场景隔离键；留空 = 全局

返回体含：`accepted`、`memory_id`、`memory_type`、`status`(accepted/rejected)、`reject_code`、`reasons`、`quality_score`、`fingerprint`、`processing_ms`、`stage_ms`、偏好相关 `preference_extracted` / `preference_keys` / `preference_stored_ids`。

> **说明：**`/ingest/file` 为 `multipart/form-data`，额外带 `caption`（内容说明）；单文件 ≤50MB，其余限制见 12.5 中间件。

12.5 /preferences/extract（偏好抽取）请求

请求体 `PreferenceExtractRequest`：`user_id`、`text`、`scene_id?`、`source`(默认 conversation)。返回 `extracted_count`、`mode`(rule/llm)、`degraded_to_rules`、`entries`（抽取结果）、`stored`（实际入库项含 memory_id / superseded_memory_id）。

> **提示：**前端「新增回答偏好」本质就是调这个接口；抽取走 9.8 的融合模式，断网自动降级规则。

12.6 /recall（检索）请求与返回

请求体 `RecallRequest`：

字段	类型	默认 / 范围	说明
user_id	string	必填	用户标识
query	string	""	检索问句；空串返回近期记忆
top_k	int	5（1–100）	返回条数上限
memory_type	enum?	null	仅查 preference / knowledge
scene_id	string?	null	场景过滤
filters	object	{}	额外过滤条件
retrieval_mode	string	auto	auto / semantic / hybrid（见 9.3.1）

返回：`count`、`latency_ms`、`strategy`(如 `query_memory:auto-semantic`)、`items`（记忆卡片列表）。

12.7 /forget（自然语言遗忘）请求与返回

请求体 `ForgetRequest`：`user_id`、`instruction`（自然语言，如「忘掉关于镜像的偏好」）。返回：`count`（实际删除数）、`deleted_ids`（真实 memory_id 列表）。后端会按占位符 / 关键词 / 类型解析目标，级联擦除短期 / 中期 / 长期 / 向量库 / 中转库，并同步清理偏好版本索引（见第十三章）。

> **注意：**全量删除需指令显式包含「所有」且限定类型才放行；普通删除有 `min_score=0.6` 置信度门控防误删。

12.8 安全加固（中间件）

机制	默认	说明
可选鉴权	关（配 `API_AUTH_TOKEN` 后开）	除 `/health` 外均需 `Authorization: Bearer <token>`，否则 401
CORS	本地前端	`API_CORS_ORIGINS` 限定来源；未配置仅放行本地且不带凭证（绝不 `*`+credentials）
限流	120 次 / 60 秒 / IP	滑动窗口超阈值返回 429
请求体上限	JSON 8MB / 文件 50MB	超 413
处理超时	180 秒（流式除外）	超 504
统一错误	—	业务异常不泄露内部细节；未捕获异常返回通用 500

12.9 /health 返回差异

- 常规：`{"status":"ok","memory_backend":...,"embedding_backend":...}`。
- 验收严格模式（kylin）：额外返回 `kylin_strict`、`using_fallback`、`vector_dim`、`vector_sdk`、`collection`、`pid`、`run_id`，且要求 `using_fallback=false`、真实 SDK 与真实向量维度匹配。

注意： `using_fallback=true` 时**不得**记为 SDK 通过；进程内存后端延迟**不得**作为 ≤500ms 验收数据。

第十三章　隐私与数据安全

隐私是这套系统的设计重点，而非附加项（对应赛题要求 (5)）。本章把「你关心的」逐项讲清机制与边界。

13.1　你要关心的五件事

你关心的方向	系统的做法
我的手机号/邮箱会不会入库	写入即脱敏为占位符，原文不落库
敏感原文存哪	AES 加密存储，密钥来自环境变量或本地密钥文件
删了还能找回吗	不能。遗忘在多库级联擦除，不可恢复
它会不会把我的隐私泄露给模型	占位符与类型保留明文用于检索，敏感原文不进模型上下文
我有高度敏感信息怎么办	脱敏不覆盖所有隐式隐私，请自行避免输入

13.2　脱敏覆盖的敏感类型

脱敏由 `security/service.py` 中**约 58 条正则**实现，覆盖 52+ 类敏感信息（满足赛题「敏感识别过滤」）。典型类别：

类别	示例
身份与证件	身份证、外国人永居证、护照、印度 Aadhaar、统一社会信用代码、出生日期
通讯	手机号（含 +86/带分隔符）、固定电话、邮箱、验证码/OTP
金融	银行卡（16–19 位，含有效期）、加密货币钱包（BTC/ETH/TRX）、
网络与设备	IPv4、MAC 地址、车牌号、SSH 私钥、设备标识
凭证与密钥	密码/口令（含 Wi-Fi）、API Key、Access/Bearer/JWT/Bot Token、Git 仓库 URL（含密码）、数据库连接串（含密码）、Tor Onion 地址

> **说明：**识别后替换为**不可逆加盐哈希占位符**（如 `<HASH_7f3a9b>`），并建立「占位符↔原文」的本地索引；仅靠占位符无法还原，还原依赖本地索引文件，实现隐私与记忆分离。

13.3　索引加密（AES-GCM）

敏感原文的「占位符↔原文」映射由 `SensitiveIndex` 持久化，且**仅原文加密**（`placeholder_id`/`type` 留明文，保证检索与统计性能不受损）：

- **算法**：AES-256-GCM（带认证标签，防篡改）。
- **密钥来源**：环境变量 `MOKOA_SECURITY_KEY` 优先（任意口令经 SHA256 派生 32 字节）；未配置则首次在 `~/.mokoa/security.key` 自动生成（非 Windows 限制 0600 权限）；文件不可写时退化为进程内随机密钥（重启后旧索引无法解密）。

- **密文格式**：`MOKOA1:<base64(nonce)>.<base64(ciphertext+tag)>`，前缀用于区分明文（兼容历史索引）与版本识别。
- **降级**：密钥变更 / 数据损坏导致解密失败时返回空串，不抛异常中断加载。

13.4 自然语言精准遗忘

遗忘由 `security/forget_parser.py` + `Orchestrator.forget` 实现：

- **解析**：把「忘掉关于我手机号的记忆」这类指令拆成**类型意图**（手机号→`phone`）+ **话题词**，两级打分（候选词元反向命中指令为核心）。
- **防误删**：置信度门控 `min_score=0.6`；**全量删除**需指令显式含「所有」且限定类型才放行。
- **标准键**：解析结果携带 `preference_key`（如 `habit.download_path`），便于前端展示与审计。
- **级联擦除**：按占位符软删敏感索引，按 memory_id 级联删除短期 / 中期 / 长期待入库 / Milvus / 中转库，并同步清理本地偏好版本索引（见 9.6、7.6）。

> **提示：**普通记忆/偏好的精准遗忘也已打通——非敏感内容（如下载路径偏好）同样能被一条自然语言指令命中删除。

13.5 上下文维护（无感压缩）

对应「上下文会越攒越长」问题，由 `core/context_perception.py` 在上下文达窗口 70–80% 时**主动**重构：

- **永不压缩**：系统提示词 / 用户提示词 / 长期记忆（偏好、系统指令、角色设定）。
- **保持不变**：最近几轮对话（当前意图核心）。
- **压缩为摘要**：中间工具调用（「已执行 3 个命令…」）、早期重复/已解决对话（「曾咨询 Python 虚拟环境，已推荐 conda」）。
- **三层策略**：关键词快速过滤 → 模型摘要 → 语义检索兜底；按 Token 压力 / 语义活跃度 / 信息密度动态选择。对用户全程无感。

13.6 中转库与保留期

ingestion 在脱敏之前写入 SQLite 中转库做审计与去重（非最终记忆库）。敏感处理：

- 原始报文默认 `redacted`（递归脱敏后归档）；`off` 不写 raw 行；`plaintext` 仅供 dev 且需双确认。
- 手动 / 自动清理默认 **7 天 raw / 90 天 audit**（自动间隔 `staging_auto_purge_interval_seconds`，默认 1h）。

• `/forget` 命中本地或 Milvus ID 后同步擦除 staging content、删除 raw 与 TTL 指纹。

13.7 多用户隔离

全链路按 `user_id` 隔离：不同用户的记忆、偏好、脱敏索引、中转库互不可见（验收用例「用户 A 遗忘不删用户 B」通过）。生产 API 必须注入真实用户标识，否则无法实现可靠隔离与精准遗忘。

13.8 P0 验收六条（自然语言遗忘）

#	验收标准	结果
①	「下载路径偏好」映射到 `habit.download_path`	☑
②	返回真实删除 ID	☑
③	偏好画像/版本索引/短中长期/向量存储同步删	☑
④	删除后 `/recall` 返回 0 条	☑
⑤	用户 A 遗忘不删用户 B	☑
⑥	审计不保留未脱敏原文	☑

13.9 一些关于边界情况的说明

• 语义级敏感识别（LLM）代码就绪但**默认关闭**，按需 `security.semantic_detection.enabled` 开启；未开时回退正则。

• 脱敏不覆盖所有隐式隐私表述（如「我住在 XX 小区」）。涉及高度敏感信息请自行避免输入。

• 测试 `tests/test_security.py` 当前 121 项 + 专项/集成 15 项，全量 741 passed。

第十四章　故障排查与常见问题

14.1　现象 → 原因 → 处理

现象	可能原因	处理
右栏一直「演示模式」、无真实数据	未连接后端	点连接按钮；确认后端已在 18080 启动、/health 正常
前端连不上后端	端口/代理失配	后端监听 18080；或改前端代理目标为实际端口
偏好不入库	保存本轮内容未开，或模型未配置	确认开关开启；检查 AGENT_API_KEY 是否填入
觉得「检索很慢」	误看总耗时	以右栏「记忆」页签的检索耗时(ms)为准，总耗时含模型生成
遗忘后好像还在	看的是旧会话缓存	遗忘会清空当前会话；重新查询记忆确认已删除
上传文件无反应	超 8MB 或类型不符	单文件≤8MB，支持 PDF/Word/常见图片；点「上传并写入记忆」才发送
处理记录显示「未接收」	质量门控或重复	看原因代码，补充更有信息量的内容再投
本地嵌入很慢/占内存	BGE-M3 本地模型较大	演示用 hash 后端；或加内存/换 kylin 后端
麒麟上 SDK 返回 fallback	未拿到真实维度	检查 OS_AGENT_KYLIN_ACCEPTANCE 与库路径，严禁用 fallback 冒充
对话无回答	模型密钥/地址错误	核对 AGENT_API_KEY/AGENT_MODEL/AGENT_API_BASE
页面打不开 5173	前端未启动或端口被占	npm run dev 重试；或换端口
切换用户数据不变	未同步	改完用户 ID 按回车或点「↻ 同步」
/ingest 返回 413	请求体超 8MB	拆分 payload；文件走 /ingest/file（上限 50MB）
/recall 偶发 504	单请求超 180s	检查模型/向量库是否卡住；流式对话不受影响

14.2　推荐排查顺序

多数问题集中在「没连后端 / 开关没开 / 端口不对」三点，按此顺序能解决 80%：

1. **点连接**：右栏状态灯变「实时同步」才算连上；演示模式操作不落库。
2. **查 health**：`curl -i http://127.0.0.1:18080/health` 期望 `200` + `{"status":"ok",...}`。
3. **对端口**：后端 `18080`、前端 `5173`；前端代理目标须指向后端实际端口。
4. **查开关**：「保存本轮内容」「使用记忆」是否开启（默认开）。
5. **核密钥**：对话无回答先核 `AGENT_API_KEY/AGENT_MODEL/AGENT_BASE_URL`。

14.3　用日志定位

• 后端日志级别由 `logging.level`（默认 INFO）控制；排障可临时调 `DEBUG` 看 ingestion 各阶段耗时（`stage_ms`）、召回策略（`strategy`）、脱敏命中。

• 中转库统计：`GET /staging/stats` 看拒绝码分布（invalid_source / empty_content / low_quality / duplicate …）与处理耗时分位，定位「为什么没入库」。

• 拒绝属于正常业务结果，不会记成 500 错误；只有未捕获异常才返回通用 500（不泄露内部细节）。

第十五章　术语表

术语	解释
MOKOA / 麒忆智脑	本项目名，意为「智能体记忆系统」
Orchestrator	记忆编排核心，统一调度摄取/召回/遗忘
偏好 Preference	用户对助手的固定要求（语言/风格/工具等）
知识 Knowledge	用户提供的经验、文档内容
场景 Scene	一次连续任务上下文；场景内偏好可隔离
召回 Recall	回答前找出相关记忆
嵌入 Embedding	把文字转成向量，供语义相似比较
短期/中期/长期	记忆按热度在三层间流转
冲突消解	新旧记忆撞车时自动裁决以谁为准
语义遗忘	用自然语言描述要删什么，系统语义匹配删除
脱敏	把敏感信息替换为占位符，原文不落库
Kylin 向量引擎	银河麒麟提供的向量存储/检索后端（验收用）
Consolidation 晋升	短期→中期→长期的自动整理与沉淀
Heat 热度	记忆被访问/复用次数的累计信号，决定能否晋升
Fingerprint 去重指纹	SHA-256 内容哈希，TTL 窗口内合并重复数据
Staging 中转库	ingestion 写入前的 SQLite 审计 + 去重库（非最终记忆库）
语义检测 Semantic Detection	用 LLM 做敏感识别（默认关，回退正则）
Recall@5	前 5 条召回中含正确答案的比例，评测召回口径
PII	个人敏感信息（Personally Identifiable Information）
AES-GCM	带认证标签的 AES 加密，用于敏感索引原文
auto-semantic	auto 模式下英文开放域问句自动走语义排序的路径

附录 A　验收与表现摘要

以下数据来自仓库评测证据与《赛题要求》《项目书符合度核查报告》，供验收与讲解引用。

A.1　赛题 7 项要求覆盖

赛题要求	对应模块	状态
(1) 多源数据整合	src/ingestion/ 五段式+SQLite 中转	已实现
(2) 偏好动态捕捉+版本+跨场景	src/preference/	已实现
(3) 知识结构化+冲突+关联检索	knowledge + conflict_resolver + recall_ranker	已实现
(4) 端侧+麒麟 Embedding+≤500ms	src/embedding/ 麒麟 SDK	已实现
(5) 敏感过滤+自然语言遗忘	src/security/	已实现
(6) 短/中期记忆流转	layer_manager/short_term/mid_term/consolidator	已实现
(7) 量化评测+测试报告	tests/(741 项)/benchmarks/scripts	已实现

A.2　四项硬指标实测

指标	赛题目标	当前实测	状态
偏好提取准确率	≥85%	封闭键 0.9429；开放域语义 0.9（n=150）	过线
知识检索召回率	≥85%	Recall@5 = 0.8707 / 147 有效问	过线
知识检索响应时间	≤500ms	MS MARCO 编排 P95 108.6~109.3ms	过线
知识冲突处理正确率	≥88%	1.00 / 142 对（更新 70+对照 72）	过线

> **备注：**数据为当前评测口径，样本规模见项目书诚实声明；开放域偏好融合+语义已达 0.89 过线。具体以仓库 evidence 与测试结果.md 为准。

A.3　评测方法与口径

指标	评测脚本 / 数据	口径说明
知识检索召回率	`scripts/eval_retrieval_datasets.py`	MS MARCO 抽样 150 问 / 2000 段，含真实随机负例（reservoir 抽样），非各问正例互扰
知识检索响应时间	orchestrator 默认编排基线	P95=386ms，`open_domain_semantic` 英文走语义排序，避开词面全表扫描
知识冲突处理正确率	`benchmarks/datasets/conflict_resolution.jsonl`（22 条）	`scripts/conflict_eval_dataset.py`，覆盖偏好漂移/手动配置权威/跨场景不冲突/知识最新事实胜出/低信任不覆盖
偏好提取准确率	`benchmarks/datasets/preference_extraction.jsonl`（35 条）	`scripts/pref_eval_dataset.py`，含原句/同义/英文/版本链/跨场景/负例；35 条负例零误报

> **说明：**所有评测均可离线复现（`benchmark.seed=42`）。历史曾出现的「召回夸大」「延迟破线」「冲突正确率缺口」「偏好无数字」四项问题均已在本次优化中修复并给出可复现口径。

A.4　关键证据文件

文件	用途
`docs/赛题要求.md`	初评评审表与四项硬指标
`docs/赛题.md` / `赛题-XA-202612…pdf`	赛题原文
`docs/ARCHITECTURE.md`	架构设计（收缩而非堆叠）
`docs/2abac5a6-信息脱敏与上下文维护.md`	脱敏 / 上下文 / 精准遗忘设计
`docs/测试结果.md`	全量测试 741+ 项
`docs/evidence/dataset-eval-20260915/`	最新评测数据集与报告
`docs/交付前核查报告_20260915-1131.md`	交付前核查

附录 B　文件与目录速览

路径	作用
api/server.py	FastAPI 服务入口，暴露全部接口（含健康检查/鉴权/限流）
src/core/	编排核心：分层、召回、冲突、上下文
├─ orchestrator.py	总调度（process_input/classify/consolidate/recall/forget）
├─ conflict_resolver.py	知识取最新事实 / 偏好信任+近因裁决
├─ recall_ranker.py	关联检索排序（语义/词面/别名融合）
├─ layer_manager.py + short_term/mid_term	短/中/长期三层与晋升
├─ context_perception.py	上下文压缩引擎
├─ classifier.py	偏好 vs 知识分类
src/ingestion/	多源摄取流水线
├─ collector/cleaner/normalizer/quality.py	四段式 + 质量评分
├─ pipeline.py	串联全流程 + SQLite 去重/审计
├─ staging_store.py	中转库（审计 + TTL 去重 + JSONL 导出）
├─ file_extract.py / vision.py	文档/图片抽取与视觉描述
src/preference/	偏好提取、版本、置信度
├─ extractor.py / store.py / keys.py / version_index.py	LLM+规则抽取 / 存储 / 键注册 / 版本链
src/knowledge/	知识检索服务
├─ memory_service/milvus_client.py / kylin_vector_client.py	向量存储（Milvus / 麒麟）
src/security/	脱敏、遗忘解析、索引加密
├─ service.py / forget_parser.py / index_crypto.py / semantic_detector.py	正则脱敏 / NL 遗忘 / AES-GCM / 语义检测
src/embedding/	统一嵌入接口（kylin/local/hash）
frontend/	Vue3 演示前端（对话+记忆双栏）
configs/default.yaml	默认配置（唯一配置文件）
docs/	架构、赛题、验收、本手册等文档
tests/ benchmarks/ scripts/	测试、跑分、评测脚本
./data/	运行期数据（ingestion_staging.db / preferences/），自动生成

附录 C　版本与版权

手册版本 v3.3，2026-09-15 发布，内容依据项目前端与后端源码核对编写（前端界面文案逐字对照 frontend/src），与真实运行行为一致。项目：麒忆智脑（MOKOA），参赛作品「挑战杯 XA-202612 揭榜挂帅 · 麒麟软件 OS Agent 记忆优化及高效应用研究」。

> **备注：**本手册为面向使用者的操作说明；架构、算法、部署细节以 README.md、ARCHITECTURE.md 及 docs/ 下技术方案为准。